



Q1 What is C language? Write its features. (4 Marks)

Year Asked: 2014, 2015, 2016, 2019, 2021, 2025

Ans:

Definition:

C is a structured, procedural, middle-level programming language. It was developed by Dennis Ritchie in 1972 at AT&T Bell Labs.

Features of C:

1. Structured language – program is divided into functions.
2. Middle level language – supports both low level and high level features.
3. Portable – can run on different hardware.
4. Fast and efficient – suitable for system programming.
5. Rich library functions – provides many built-in functions.
6. Case sensitive – 'A' and 'a' are different.
7. Extensible – new functions can be added.



Dennis Ritchie is known as the Father of C.

Q2 Write the structure of a C program. (4 Marks)

Year Asked: 2014, 2016, 2018, 2019, 2022

Ans:

A C program has a fixed structure. It consists of the following parts:

1. **Documentation Section** – used for comments.
2. **Link Section** – header files are included.
3. **Definition Section** – symbolic constants are defined.
4. **Global Declaration Section** – global variables and function prototypes are declared.
5. **main() Function** – program execution starts from here.
6. **User Defined Functions** – additional functions are written after main().

Syntax / Format:

```
/* Documentation Section */
#include <stdio.h>           // Link Section
#define PI 3.14             // Definition Section
int global;                 // Global Declaration Section

int main()                  // main() Function
{
    // statements
    return 0;
}

// User Defined Function(s)
void display()
{
    // statements
}
```

Remember
this format
and write
in exam
exactly like
this.

Q3 Explain data types in C with size and range. (4 Marks)

Year Asked: 2014, 2017, 2019, 2021, 2022

Ans:

Data type specifies the type of data that a variable can store in C. Different data types with their size and range are given below:

Data Type	Size (Bytes)	Range / Description
int	4	-2,147,483,648 to +2,147,483,647
float	4	6 – 7 decimal digits
double	8	15 – 16 decimal digits
char	1	-128 to 127
void	—	No value

Type Modifiers: short, long, signed, unsigned

Derived Data Types: array, pointer, function

User Defined Data Types: struct, union, enum, typedef



Use correct data type as per requirement to save memory and get accurate range.

Q4 Explain different types of operators in C with examples. (4 Marks)

Year Asked: 2015, 2016, 2019, 2021, 2023

Ans:

Operators are special symbols used to perform operations on data. The types of operators in C are:

Type	Operators	Example
1. Arithmetic	+ - * / %	a + b, a % b
2. Relational	== != > < >= <=	a > b
3. Logical	&& !	(a > 0 && b > 0)
4. Assignment	= += -= *= /= %=	a += 5
5. Unary	++ -- + - !	a++, --a
6. Conditional (Ternary)	? :	max = (a > b) ? a : b;
7. Bitwise	& ^ ~ << >>	a & b
8. Special	sizeof() , , & *	sizeof(int)



Write name, symbols and one example for each operator to get full marks.

**Q1** What is a function? Explain the types of functions in C. (4 Marks)

Year Asked: 2014, 2016, 2019, 2021, 2024

Ans:

A function is a block of statements that performs a specific task. It helps to break a large program into smaller modules. In C, a function is called using its name.

Types of Functions in C:

- 1. Library (Built-in) Functions** – Provided by C compiler, e.g., printf(), scanf(), strlen().
- 2. User Defined Functions** – Written by the programmer as per requirement.
User defined functions are further divided into:
 - Functions with no arguments and no return value.
 - Functions with arguments but no return value.
 - Functions with no arguments but return value.
 - Functions with arguments and return value.



Functions improve reusability, readability and modularity of a program.

Q3 Explain passing arguments to functions in C. Differentiate between call by value and call by reference. (4 Marks)

Year Asked: 2014, 2016, 2019, 2020, 2023

Ans:

Arguments are the actual values passed to a function when it is called. They are received by parameters in the function definition.

Feature	Call by Value	Call by Reference
Method	Passes actual value	Passes address (using pointers)
Effect on actual arguments	No change	Changes value
Memory usage	Separate copy is used	Uses same memory location
Syntax	func(int a)	func(int *a)
Example	void square(int a) { a = a * a; }	void square(int *a) { *a = (*a) * (*a); }



Call by value is safe, while call by reference is used when we need to modify the original value.

Q2 Explain function declaration, function definition and function call with example. (4 Marks)

Year Asked: 2015, 2017, 2019, 2021, 2022

Ans:

- 1. Function Declaration (Prototype):** It tells the compiler about the function name, return type and parameters.

Syntax: return_type function_name(parameter_list);**Example:** int sum(int, int);

- 2. Function Definition:** It contains the actual code or statements to perform the task.

Syntax:

```
int sum(int a, int b)    // function definition
{
    return a + b;
}
```

- 3. Function Call:** It is used to invoke the function.

Example:

```
int main()
{
    int s = sum(10, 20);    // function call
}
```



Declaration tells, definition does, and call uses the function.

Q4 Explain the use of return statement in C functions with example. (4 Marks)

Year Asked: 2015, 2017, 2021, 2023, 2025

Ans:

The return statement is used to send a value back to the calling function. It terminates the function and transfers control to the calling function.

Syntax: return expression;

Example: Write a function to find the sum of two numbers using return statement.

```
#include <stdio.h>
int sum(int a, int b)
{
    return a + b;    // return statement
}
int main()
{
    int x = 10, y = 20;
    int s = sum(x, y);    // function call
    printf("Sum = %d", s);
    return 0;
}
```



Use return to give result from a function to the calling function.



Q1 What is an array? Explain its types. (4 Marks)

Year Asked: 2014, 2015, 2018, 2020, 2022, 2024

Ans:

An array is a collection of similar data types stored in contiguous (adjacent) memory locations. All the elements of an array are accessed using a common name and an index (subscript). Indexing in C starts from 0.

Syntax:

```
data_type array_name[size];
```

Types of Arrays in C:

1. **One-Dimensional Array (1D)** – linear list of elements.
2. **Two-Dimensional Array (2D)** – elements in row and column form (matrix).
3. **Multi-Dimensional Array** – array with more than two dimensions (e.g., 3D).
4. **Character Array (String)** – array of characters terminated by a null character ('\0').
5. **Array of Pointers** – array whose elements are pointers.



Array is used to store multiple values of the same type in a single variable name.

Q3 What is a two-dimensional array? How are elements stored and accessed? Explain with example. (4 Marks)

Year Asked: 2015, 2017, 2019, 2021, 2022, 2024

Ans:

A two-dimensional array is an array of arrays arranged in row and column form. It is used to store data in a matrix form. Elements are stored in row-major order (row by row) in contiguous memory locations.

Syntax:

```
data_type array_name[row][column];
```

Example: `int a[2][3];` // 2 rows and 3 columns.

Accessing Elements:

Each element is accessed using two indices: `a[i][j]`

Example Program:

```
#include <stdio.h>
int main()
{
    int i, j, a[2][3];
    printf("Enter elements:\n");
    for(i=0; i<2; i++)
        for(j=0; j<3; j++)
            scanf("%d", &a[i][j]);
    printf("\nMatrix is:\n");
    for(i=0; i<2; i++)
    {
        for(j=0; j<3; j++)
            printf("%d ", a[i][j]);
        printf("\n");
    }
    return 0;
}
```

Memory Storage (Row-major order)

a[0][0]	a[0][1]	a[0][2]
a[1][0]	a[1][1]	a[1][2]

Stored as:

`a[0][0], a[0][1], a[0][2], a[1][0], a[1][1], a[1][2]`



Use two indices to access elements of a 2D array.

Q2 Write a program to input and display elements of a one-dimensional array. (4 Marks)

Year Asked: 2014, 2016, 2019, 2021, 2023

Ans:

```
#include <stdio.h>
int main()
{
    int n, i, a[100];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("\nElements of array are:\n");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Sample Output:

```
Enter number of elements: 5
Enter 5 elements:
10 20 30 40 50

Elements of array are:
10 20 30 40 50
```



Use for loop and index to access array elements.

Q4 Explain character array and string handling functions in C. (4 Marks)

Year Asked: 2014, 2016, 2019, 2021, 2023

Ans:

A character array is an array of characters used to store a string. In C, a string is a sequence of characters terminated by a null character ('\0').

Syntax: `char str[size];`

Example: `char str[20] = "Hello";`

Common String Handling Functions (in <string.h>):

Function	Use
<code>strlen()</code>	Returns length of string (excluding '\0')
<code>strcpy()</code>	Copies one string to another
<code>strcat()</code>	Concatenates two strings
<code>strcmp()</code>	Compares two strings (0 if equal)
<code>strrev()</code>	Reverses a string (some compilers)
<code>strupr()</code>	Converts string to uppercase (some compilers)
<code>strlwr()</code>	Converts string to lowercase (some compilers)

Example Program (strlen):

```
#include <stdio.h>
#include <string.h>
int main()
{
    char str[50];
    printf("Enter a string: ");
    gets(str);
    printf("Length of string = %d", strlen(str));
    return 0;
}
```



Always include <string.h> to use string functions.

Q1 What is a pointer? Explain with example. (4 Marks)

Year Asked: 2014, 2016, 2018, 2020, 2022, 2024

Ans:

A pointer is a variable that stores the memory address of another variable. It is declared using the asterisk (*) operator. Pointers are used to access and manipulate data indirectly using their addresses.

Syntax:

```
data_type *pointer_name;
```

Memory Representation

```

a
(10) → 2000

p
(2000) → 3000
    
```

Example:

```
#include <stdio.h>
int main()
{
    int a = 10;
    int *p = &a; // p stores address of a
    printf("Value of a = %d\n", a);
    printf("Address of a = %p\n", &a);
    printf("Value of p = %p\n", p);
    printf("Value using pointer = %d\n", *p);
    return 0;
}
```



A pointer stores address, not the actual value.

Q2 Explain pointer arithmetic with example. (4 Marks)

Year Asked: 2015, 2017, 2019, 2021, 2023

Ans:

Pointer arithmetic means performing arithmetic operations on pointers. We can increment (++), decrement (--) or add a number to a pointer. When we increment a pointer, it moves to the next memory location based on the size of the data type.

Example Program:

```
#include <stdio.h>
int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;

    printf("%d\n", *p);
    p++; // points to next int (20)
    printf("%d\n", *p);
    p = p + 2; // skip two locations
    printf("%d\n", *p); // 40
    return 0;
}
```

Key Points:

- p++ moves to next location
- p-- moves to previous location
- p + n moves n locations forward
- p - n moves n locations backward
- pointers can be added or subtracted with integers only



In pointer arithmetic, the pointer increases/decreases by the size of the data type (e.g., 4 bytes for int).

Q3 Explain call by value vs call by reference using pointers in C. (4 Marks)

Year Asked: 2014, 2018, 2020, 2022, 2025

Ans:

In call by value, a copy of the actual value is passed to the function, so changes inside the function do not affect the original variable. In call by reference (using pointers), the address of the variable is passed, so changes inside the function affect the original variable.

Call by Value (No Change)

```
#include <stdio.h>
void square(int x) // call by value
{
    x = x * x;
}
int main()
{
    int a = 5;
    square(a);
    printf("Value of a = %d", a);
    return 0;
}
```

Call by Reference (Using Pointers)

```
#include <stdio.h>
void square(int *x) // call by ref
{
    *x = (*x) * (*x);
}
int main()
{
    int a = 5;
    square(&a);
    printf("Value of a = %d", a);
    return 0;
}
```

Output:

Value of a = 5 (no change)

Output:

Value of a = 25 (changed)



Use pointers (call by reference) when you want to modify the original value of a variable.

Q4 Write a program to swap two numbers using pointers. (4 Marks)

Year Asked: 2015, 2017, 2019, 2023, 2025

Ans:

Program:

```
#include <stdio.h>
void swap(int *a, int *b)
{
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
int main()
{
    int x = 10, y = 20;
    printf("Before swap: x = %d, y = %d\n", x, y);
    swap(&x, &y); // pass addresses
    printf("After swap: x = %d, y = %d\n", x, y);
    return 0;
}
```

Sample Output:

Before swap: x = 10, y = 20
After swap: x = 20, y = 10

Key Points:

- Addresses of variables are passed to the function.
- Values are swapped using dereferencing (*).
- Original values get changed.



Use pointers to swap values without using global variables or returning multiple values.



Q1 What is a structure in C? Explain with example. (4 Marks)

Year Asked: 2014, 2016, 2019, 2021, 2023, 2025

Ans:

A structure is a user defined data type in C that allows us to store different data types under a single name. It is used to represent a record.

Syntax:

```
struct structure_name
{
    data_type member1;
    data_type member2;
    ...
};
```

Example:

```
#include <stdio.h>
struct student
{
    int roll;
    char name[50];
    float marks;
};
int main()
{
    struct student s = {1, "Ali", 85.5};
    printf("Roll: %d\nName: %s\nMarks: %.2f",
           s.roll, s.name, s.marks);
    return 0;
}
```

Output:

```
Roll: 1
Name: Ali
Marks: 85.50
```



Structure is used to store related data items.

Q2 Explain how to access structure members. Write a program. (4 Marks)

Year Asked: 2015, 2017, 2020, 2022, 2024

Ans:

Structure members are accessed using the dot (.) operator with a structure variable.

Syntax:

```
structure_variable.member_name;
```

Example Program:

```
#include <stdio.h>
struct book
{
    int id;
    char title[50];
    float price;
};
int main()
{
    struct book b;
    b.id = 101;
    strcpy(b.title, "C Programming");
    b.price = 250.50;

    printf("Book ID: %d\n", b.id);
    printf("Title: %s\n", b.title);
    printf("Price: %.2f\n", b.price);
    return 0;
}
```

Output:

```
Book ID: 101
Title: C Programming
Price: 250.50
```

Key Points:

- Use dot (.) operator.
- Each member is accessed using variable_name.member.
- Can read or modify values.
- Used in real applications like student record, employee record etc.

Q3 What is a union in C? Explain with example. (4 Marks)

Year Asked: 2014, 2018, 2021, 2023

Ans:

A union is a user defined data type in C, similar to structure, but in a union all members share the same memory location. Only one member can hold a value at a time.

Syntax:

```
union union_name
{
    data_type member1;
    data_type member2;
    ...
};
```

Example:

```
#include <stdio.h>
union data
{
    int i;
    float f;
    char ch;
};
int main()
{
    union data d;
    d.i = 10;
    printf("Value of i = %d\n", d.i);
    d.f = 3.14;
    printf("Value of f = %.2f\n", d.f);
    return 0;
}
```

Output:

```
Value of i = 10
Value of f = 3.14
```

Key Points:

- All members share same memory location.
- Size of union = size of largest member.
- Only one member can store value at a time.
- Used to save memory.

Q4 What is an enum in C? Explain with example. (4 Marks)

Year Asked: 2015, 2019, 2022, 2024

Ans:

An enum (enumeration) is a user defined data type in C which consists of a set of named integer constants. It makes the program more readable.

Syntax:

```
enum enum_name
{
    constant1,
    constant2,
    constant3
};
```

Output:

```
Value of WED = 3
```

Key Points:

- By default, first constant = 0, next = 1 and so on.
- Improves readability of program.
- Used for fixed set of related constants.
- We can also assign custom values.



Enum is useful when a variable can take only certain fixed values.



Q1 What is a structure? Explain its syntax with an example. (4 Marks)

Year Asked: 2014, 2016, 2019, 2021, 2023, 2025

Ans:

A structure is a user defined data type in C that allows different data types to be stored under a single name. It is used to represent a real world entity. Each member of a structure is called a structure variable or member.

Syntax:

```
struct structure_name
{
    data_type member1;
    data_type member2;
    ....;
};
```

Example:

```
struct student
{
    int roll;
    char name[50];
    float marks;
};
```

Key Points:

- Structure is a collection of different data types.
- Members can be of any data type.
- Memory is allocated for all members together.
- Accessed using dot (.) operator.

Q2 Write a program to input and display information of a student using structure. (4 Marks)

Year Asked: 2015, 2017, 2019, 2022, 2024

Ans:

```
#include <stdio.h>
struct student
{
    int roll;
    char name[50];
    float marks;
};

int main()
{
    struct student s;
    printf("Enter roll number: ");
    scanf("%d", &s.roll);
    printf("Enter name: ");
    scanf("%s", s.name);
    printf("Enter marks: ");
    scanf("%f", &s.marks);
    printf("\nRoll No: %d", s.roll);
    printf("\nName : %s", s.name);
    printf("\nMarks : %.2f", s.marks);
    return 0;
}
```

Sample Input:

Enter roll number: 101
Enter name: Ali
Enter marks: 85.5

Sample Output:

Roll No: 101
Name : Ali
Marks : 85.50

Key Points:

- Use dot (.) operator to access members.
- Structure can store different data types.
- Useful for handling real world data.

Q3 Explain how to access and initialize structure members. Give example. (4 Marks)

Year Asked: 2014, 2018, 2020, 2023

Ans:

Structure members are accessed using dot (.) operator. They can be initialized at the time of declaration or later using the structure variable name and dot operator.

Example 1: Initialize at the time of declaration

```
#include <stdio.h>
struct student
{
    int roll;
    char name[50];
    float marks;
};

int main()
{
    struct student s = {101, "Sana", 90.5};
    printf("%d %s %.2f", s.roll, s.name, s.marks);
    return 0;
}
```

Other Ways of Initialization:

- At declaration
- Using assignment
- Using input from user
- Using strcpy() for strings

Output:

101 Sana 90.50



Use dot (.) operator to access or initialize structure members.

Q4 Differentiate between structure and array in C. (4 Marks)

Year Asked: 2015, 2019, 2021, 2022, 2024

Ans:

Basis	Structure	Array
Definition	Collection of different data types under one name.	Collection of similar data types under one name.
Data types	Can store different data types.	Can store only same data type.
Access	Members accessed using dot (.) operator.	Elements accessed using index [].
Memory	Memory is allocated for all members together.	Memory is allocated for similar elements.
Use	Used to represent a real world entity.	Used to store multiple values of same type.
Example	struct student { int roll; char name[50]; }	int a[5]; // stores 5 integers

Key Points:

- Structure → different data types, accessed by dot (.)
- Array → same data type, accessed by index []
- Both are user defined data structures (array is derived type).

Q1 What is a structure in C? Explain with example. (4 Marks)

Year Asked: 2014, 2016, 2019, 2021, 2023, 2025

Ans:

A structure is a user defined data type in C that allows us to group different data types under a single name. It is used to store related information.

Syntax:

```
struct structure_name
{
    data_type member1;
    data_type member2;
    ...;
};
```

Example Program:

```
#include <stdio.h>
struct student
{
    int roll;
    char name[50];
    float marks;
};
int main()
{
    struct student s = {1, "Ali", 85.5};
    printf("Roll: %d\n", s.roll);
    printf("Name: %s\n", s.name);
    printf("Marks: %.2f\n", s.marks);
    return 0;
}
```

Key Points:

- Structure groups different data types.
- Declared using struct keyword.
- Members are accessed using dot (.) operator.
- Saves related data together.

Output:

Roll: 1
Name: Ali
Marks: 85.50

Q2 Explain nested structure with example. (4 Marks)

Year Asked: 2015, 2018, 2020, 2022, 2024

Ans:

A structure within another structure is called a nested structure. It is used when one structure contains another structure as a member.

Example Program:

```
#include <stdio.h>
struct date
{
    int day;
    int month;
    int year;
};
struct student
{
    int roll;
    char name[50];
    struct date dob; // nested structure
};
int main()
{
    struct student s;
    s.roll = 1;
    strcpy(s.name, "Sana");
    s.dob.day = 12;
    s.dob.month = 8;
    s.dob.year = 2005;

    printf("Roll: %d\n", s.roll);
    printf("Name: %s\n", s.name);
    printf("DOB: %d-%d-%d\n", s.dob.day, s.dob.month, s.dob.year);
    return 0;
}
```

Key Points:

- Structure can have another structure as a member.
- Used to represent complex data.
- Access members using dot (.) operator.
- Useful in real-life applications like student records, employee details etc.

Output:

Roll: 1
Name: Sana
DOB: 12-8-2005

Q3 What is a union in C? Explain with example. (4 Marks)

Year Asked: 2014, 2017, 2019, 2021, 2023

Ans:

A union is a user defined data type in C that allows us to store different data types in the same memory location. In a union, all members share the same memory, so only one member can hold a value at a time.

Syntax:

```
union union_name
{
    data_type member1;
    data_type member2;
    ...;
};
```

Example Program:

```
#include <stdio.h>
union data
{
    int i;
    float f;
    char ch;
};
int main()
{
    union data d;
    d.i = 10;
    printf("Integer: %d\n", d.i);
    d.f = 3.14;
    printf("Float: %.2f\n", d.f);
    d.ch = 'A';
    printf("Character: %c\n", d.ch);
    return 0;
}
```

Key Points:

- Union members share the same memory location.
- Only one member can hold a value at a time.
- Declared using union keyword.
- Size of union is equal to the size of its largest member.
- Useful for memory optimization.

Memory Representation:



Output:

Integer: 10
Float: 3.14
Character: A

Q4 Differentiate between structure and union. (4 Marks)

Year Asked: 2015, 2018, 2020, 2022, 2025

Ans:

The main differences between structure and union are:

S. No.	Structure	Union
1.	Each member has its own memory location.	All members share the same memory location.
2.	Size is the sum of sizes of all members.	Size is equal to the size of the largest member.
3.	All members can hold values at the same time.	Only one member can hold a value at a time.
4.	Changes in one member do not affect other members.	Changing one member affects all members (since they share memory).
5.	Used when we need to store all related data.	Used when we need to save memory and only one value is required at a time.
6.	Example: student record with roll, name, marks.	Example: store integer or float or character in same memory.



Use structure when you want to store all related information.
Use union when you want to save memory and use only one value at a time.



Read PYQ



Practice More



Write Exact



Score High

AKU PATNA | BCA-204 | Unit 5

Study
Smart
Write Less
Remember More
Score Higher !

MICRO-CHEATSHEET : UNIT - 6

(C Preprocessor)

"Preprocessor
works before
the actual
compilation."

- ✓ Introduction to Preprocessor
- ✓ Preprocessor Directives
- ✓ Macro Substitution
- ✓ File Inclusion
- ✓ Conditional Compilation
- ✓ Other Preprocessor operators

Q1. What is C preprocessor? Discuss the various preprocessor directives available in C with suitable examples.

[12 Marks]

Ans :**1. Introduction / Definition :**

- The C preprocessor is a program that processes the source code before actual compilation.
- It handles the preprocessor directives which start with '#'. It performs tasks like macro substitution, file inclusion and conditional compilation.
- The output of preprocessor is a modified source code which is then given to the compiler.

2. Common Preprocessor Directives :

Directive	Purpose	Example
#define	Defines a macro/constant	#define PI 3.14
#include	Includes a header file	#include <stdio.h>
#undef	Undefines a macro	#undef PI
#if	Conditional compilation	#if x > 0
#ifdef	Checks if macro is defined	#ifdef PI
#ifndef	Checks if macro is not defined	#ifndef MAX
#else	Else part in condition	#else
#elif	Else if part	#elif x == 0
#endif	Ends conditional compilation	#endif
#pragma	Provides additional information	#pragma warning
#line	Changes line number	#line 100
#error	Generates an error message	#error "Error here"

3. Example - Macro and File Inclusion :

```
#include <stdio.h> // file inclusion
#define PI 3.14 // macro definition
int main()
{
    float r, area;
    printf("Enter radius: ");
    scanf("%f", &r);
    area = PI * r * r; // macro substitution
    printf("Area = %.2f", area);
    return 0;
}
```

Output :

Enter radius: 5
Area = 78.50

4. Conditional Compilation Example :

```
#include <stdio.h>
#define DEBUG
int main()
{
    #ifdef DEBUG
        printf("Debug mode ON\n");
    #else
        printf("Debug mode OFF\n");
    #endif
    return 0;
}
```

Output :

Debug mode ON

Memory Trick :

#if → If
#else → Else
#endif → End If

5. Conclusion :

- The C preprocessor makes the program flexible, modular and portable by providing various directives for code substitution, file inclusion and conditional compilation.

Q2. What is macro substitution? Explain the types of macros with examples. Also write the difference between #include <filename> and #include "filename".

[12 Marks]

Ans :**1. Macro Substitution :**

- Macro substitution is the process of replacing a macro definition with its value before compilation.
- It is performed using #define directive.
- It helps to make the code shorter and easier to maintain.

2. Types of Macros :**(i) Object-like Macro (Constant Macro) :**

- Used to define a constant value.
- Syntax : #define name value
- Example : #define MAX 100
After substitution : MAX → 100

(ii) Function-like Macro :

- Used to define a macro with arguments.
- Syntax : #define name(arguments) expression
- Example : #define SQUARE(x) ((x) * (x))
After substitution : SQUARE(5) → ((5) * (5))

3. Examples :

```
#include <stdio.h>
#define PI 3.14 // object-like macro
#define AREA(r) (PI * r * r) // function-like macro
int main()
{
    int r = 5;
    float a = AREA(r);
    printf("Area = %.2f", a);
    return 0;
}
```

Output :

Area = 78.50

Memory Trick :

< > → System file
" " → User file

4. Difference between #include <filename> and #include "filename" :

Basis	#include <filename>	#include "filename"
Purpose	Used for standard library files	Used for user defined files
Search location	Searches in system include directory	Searches in current directory first, then system directory
Example	#include <stdio.h>	#include "myfile.h"

5. Conclusion :

- Macro substitution and file inclusion are important features of C preprocessor which help in code reusability and modular programming.

Study
Smart
Write Exact
Score High !

MICRO-CHEATSHEET : UNIT - 1**(Fundamentals of C)****Unit-1 Topics :**

- C Language
- Data types
- Constants & Variables
- Operators & Expressions
- Structure of C Program

Q1. What is C language? Explain its features and write the structure of a C program with suitable example.

[12 Marks]

Ans :**1. Introduction / Definition :**

→ C is a structured, procedural and general purpose programming language. It was developed by Dennis Ritchie in 1972 at AT & T Bell Laboratories. It is known as a middle level language because it supports both low level and high level programming features.

2. Features of C Language :

- ① Simple and Structured - easy to learn and use.
- ② Middle level language - supports low level and high level features.
- ③ Portable - can run on different hardware.
- ④ Fast and Efficient - provides high performance.
- ⑤ Rich set of Operators - large standard library.
- ⑥ Modular - uses functions, easy to maintain.
- ⑦ Case Sensitive - 'A' and 'a' are different.
- ⑧ Extensible - new functions can be added.
- ⑨ Widely used - in system programming, compilers, OS, embedded systems etc.

5. Conclusion :

→ C is a powerful and flexible language. Its simple structure and rich features make it the foundation of many modern programming languages.

PYQ ALERT : REPEATED / MOST VVI

Asked in Years : 2014, 2015, 2016, 2019, 2021, 2025
(Same /related question)

Marks : 12

3. Structure of a C Program :**4. Example of a C Program :**

```
#include <stdio.h> // Link section
#define PI 3.14 // Definition section
int main() // main function
{
    int r;
    float area;
    printf("Enter radius: ");
    scanf("%d", &r);
    area = PI * r * r;
    printf("Area = %.2f", area);
    return 0;
}
```

Output :

Enter radius: 5
Area = 78.50

Key Point :

main() is the starting point of every C program.

Q2. Explain different data types in C with their size and range. Also explain constants and variables with suitable examples.

[12 Marks]

Ans :**1. Introduction :**

→ Data type tells the compiler what kind of data a variable can store. Each data type has a different memory size and range of values.

2. Basic Data Types in C :

Data Type	Size (Bytes)	Range / Description
char	1	-128 to 127
int	4	-2,147,483,648 to 2,147,483,647
float	4	6-7 decimal digits
double	8	15-16 decimal digits
void	-	No value

3. Type Modifiers :

short, long, signed, unsigned

Example : long int (8 bytes), unsigned int (0 to 4,294,967,295)

4. Derived Data Types :

array, pointer, function

5. User Defined Data Types :

structure, union, enum, typedef

PYQ ALERT : REPEATED / MOST VVI

Asked in Years : 2014, 2015, 2017, 2019, 2021, 2022
(Same /related question)

Marks : 12

6. Constants in C :

→ A constant is a fixed value that does not change during program execution.

→ Types :

- Integer constant : 10, -25
- Floating constant : 3.14, -0.5
- Character constant : 'A', '9'
- String constant : "Hello"
- Symbolic constant : #define PI 3.14

Example :

```
#define PI 3.14
const int MAX = 100;
```

7. Variables in C :

→ A variable is a named memory location used to store data which can change during program execution.

Example :

```
int a = 10;
float b = 5.5;
char ch = 'A';
```

Remember :

Constant = Fixed value

Variable = Value can change

8. Conclusion :

Proper use of data types, constants and variables helps in efficient memory management and error free programming.

Study
Smart
Write Exact
Score High !

MICRO-CHEATSHEET : UNIT - 2

(Functions in C)

Unit-2 Topics :

- ✓ Definition and use of functions
- ✓ Types of functions
- ✓ Function declaration & calling
- ✓ Passing arguments to functions
- ✓ Return values
- ✓ Scope of variables
- ✓ Recursion
- ✓ Storage classes (auto, extern, static, register)

Q1. What do you mean by function? How we can use the user defined functions? Explain with a suitable example program. [12 Marks]

Ans :

1. Definition :

- A function is a block of statements that performs a specific task.
- It helps to break a large program into smaller modules.
- A function can be a library function or a user defined function.

2. Types of Functions :

- (i) Library function : Predefined functions (e.g. printf(), scanf())
- (ii) User defined function : Functions created by the user as per requirement.

3. Advantages of Functions :

- Modularity - divide program into smaller parts.
- Reusability - same function can be used many times.
- Reduces code size and complexity.
- Easy to debug and maintain.
- Helps in team programming.

4. User Defined Function - Syntax :

Function
definition
(called function)

```
return_type function_name(parameters)
{
    // local declarations
    statements;
    return value; // (if any)
}
```

Function call
(calling function)

```
function_name(arguments);
```

5. Example Program : (Add two numbers using user defined function)

```
#include <stdio.h>
int add(int a, int b) // function definition
{
    return a + b;
}
int main() // calling function
{
    int x, y, sum;
    printf("Enter two numbers: ");
    scanf("%d %d", &x, &y);
    sum = add(x, y); // function call
    printf("Sum = %d\n", sum);
    return 0;
}
```

Sample Output :

```
Enter two numbers: 5 7
Sum = 12
```

6. Conclusion :

- Functions make the program simple, structured and easy to maintain.
- User defined functions are very useful for solving large problems.

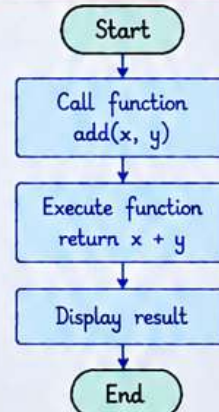
Memory Trick : "Function = F → Focus on one task."

PYQ ALERT : REPEATED / MOST VVI

Asked in Years : 2014, 2015, 2016, 2017, 2019, 2021
(Same / related question)

Marks : 4 (in 2015), can be 12 marks)

Flowchart (Function Call)



Note :

Control goes to function, executes and returns value to calling function.

Q2. What is a storage class? Discuss the various storage classes in C. [12 Marks]

Ans :

1. Definition :

- A storage class specifies the scope, lifetime and default value of a variable in C.
- It tells the compiler where the variable is stored and how long it exists.

2. Types of Storage Classes :

Storage Class	Scope	Lifetime	Default Value	Example
auto	Local	Within block	Garbage	auto int x; (int x;)
register	Local	Within block	Garbage	register int x;
static	Local	Throughout program	0	static int x;
extern	Global	Throughout program	0	extern int x;

3. Explanation :

- ① **auto** : Default storage class for local variables. Scope is within the block. Lifetime is till the block ends.
2. **register** : Similar to auto but suggests storing variable in CPU register for faster access.
3. **static** : Value remains preserved between function calls. Default value is 0.
4. **extern** : Used to access global variable declared in another file. Default value is 0.

4. Conclusion :

- Storage classes control the visibility, lifetime and memory allocation of variables in a program.
- Proper use of storage classes helps in efficient memory management.

Memory Trick : "A R S E → Auto, Register, Static, Extern"

Study Smart
Write Exact
Score High !

MICRO-CHEATSHEET : UNIT - 3**(Arrays and Strings)****Unit-3 Topics :**

- Defining and processing arrays
- Passing arrays to functions
- Multi-dimensional arrays
- Arrays and Strings

Q1. What is an array? What is a two-dimensional array? How 2-D arrays are initialized? Also explain how array elements are accessed in a 2-D array. Give example. [12 Marks]

PYQ ALERT : MOST REPEATED

Asked in Years : 2015 (Group B, Q2(c))
(Same /related topic in other years - Year not verified)
Marks : 12 (in 2015 paper, this sub-question carried 4 marks, but full 12 marks question on arrays is highly expected)

Ans :**1. Array :**

- An array is a collection of elements of the same data type stored in contiguous memory locations.
- All elements are accessed using a single name and index.
- Index always starts from 0 in C.

2. One-dimensional Array :

- **Syntax :** data_type array_name[size];
- Example :** int a[5] = {10, 20, 30, 40, 50};

3. Two-dimensional Array :

- A two-dimensional array is a collection of elements arranged in rows and columns (matrix form).
- It can be considered as an array of one-dimensional arrays.
- **Syntax :** data_type array_name[rows][columns];
- Example :** int a[2][3]; // 2 rows, 3 columns

4. Initialization of 2-D Array :

(i) During declaration :

```
int a[2][3] = { {1, 2, 3},
               {4, 5, 6} };
```

(ii) Row wise initialization :

```
int a[2][3] = {1,2,3,4,5,6};
```

(iii) Partial initialization :

```
int a[2][3] = { {1,2}, {4} };
```

Memory Layout (2 × 3 Array)			
	0	1	2
0	1	2	3
2	4	5	6
	↑	↑	
	Row index (i)	Column index (j)	

5. Accessing Elements :

- Each element is accessed using row and column index.
- **Syntax :** array_name[i][j];

Example :

```
int a[2][3] = { {1, 2, 3},
               {4, 5, 6} };
printf("%d", a[1][2]); // Output : 6
```

6. Conclusion :

- Arrays (1-D and 2-D) help to store multiple values of same type. 2-D arrays are useful for matrix and tabular data. Elements are accessed using row and column index.

Memory Trick : "2-D Array = Table (Row, Column) → a[i][j]"

Q2. What is a string in C? How strings are represented and processed? Write the commonly used string handling functions with examples. [12 Marks]

PYQ ALERT : REPEATED / MOST VVI

Asked in Years : 2014, 2016, 2019, 2021, 2022
(Same /related topic)
Marks : 12

Ans :**1. String :**

- A string is a character array used to store a sequence of characters.
- In C, a string is represented as an array of characters terminated by a null character '\0'.
- **Example :** char str[6] = "Hello"; // H e l l o \0

2. Representation in memory :

H	e	l	l	o	\0
0	1	2	3	4	5

Null character ('\\0') marks the end of string

3. String Input / Output :**Input :**

```
char str[20];
scanf("%s", str); // single word
```

Output :

```
printf("%s", str);
```

Note : For multi-word input use gets() or fgets()

4. Common String Handling Functions (<string.h>) :

Function	Purpose	Example
strlen()	Returns length of string	strlen(str)
strcpy()	Copies one string to another	strcpy(s1, s2)
strcat()	Concatenates two strings	strcat(s1, s2)
strcmp()	Compares two strings	strcmp(s1, s2)
strrev()	Reverses a string	strrev(str)
strupr()	Converts to upper case	strupr(str)
strlwr()	Converts to lower case	strlwr(str)

5. Example Program : (Find length and concatenate)

```
#include <stdio.h>
#include <string.h>
int main()
{
    char s1[20] = "Hello";
    char s2[20] = "World";
    printf("Length = %d\n", strlen(s1));
    strcat(s1, s2);
    printf("After concat : %s\n", s1);
    return 0;
}
```

Output :

```
Length = 5
After concat :
HelloWorld
```

String functions are in <string.h>

6. Conclusion :

- Strings are character arrays terminated by '\0' and can be manipulated using built-in string handling functions.

Memory Trick : "String ends with '\\0' → Always include <string.h>"

Study
Smart
Write Exact
Score High !

MICRO-CHEATSHEET : UNIT - 3**(Arrays and Strings)****Unit-3 Topics :**

- ✓ Defining and processing arrays
- ✓ Passing arrays to functions
- ✓ Multi-dimensional arrays
- ✓ Arrays and strings

Q1. What is an array? What is a two-dimensional array? How two-dimensional arrays are initialized? Also explain how array elements are accessed in a two-dimensional array [12 Marks] with a suitable example program.

Ans :

1. Array :

- An array is a collection of similar data items stored in contiguous memory locations under a single name.
- It is used to store multiple values of the same type.
- Each element is accessed using an index (subscript).
- Index always starts from 0 in C.

2. Two-Dimensional Array :

- A two-dimensional array is an array of arrays. It can be considered as a table with rows and columns.
- It is declared as: `data_type array_name[row][column];`
- It is used to store data in matrix form.

3. Initialization of 2-D Array :

- We can initialize a 2-D array at the time of declaration.
- **Example :**

```
int a[2][3] = { {1, 2, 3},
               {4, 5, 6} };
```

4. Accessing Elements :

- Each element is accessed using two indices: `array_name[row][column]`
- **Example :** `a[1][2]` is the element of 2nd row and 3rd column.

PYQ ALERT : REPEATED / MOST VVI

Asked in Years : 2015 (Q2c)

(Same / related question may appear again)

Marks : 12 (in some years full question), 4 (in 2015)

Memory Representation of 2-D Array (2 x 3)

Column →	0	1	2
Row ↓	a[0][0]	a[0][1]	a[0][2]
1	a[1][0]	a[1][1]	a[1][2]

`a[i][j]`
i = row
j = column

5. Example Program : (Input and Display 2-D Array)

```
#include <stdio.h>
int main()
{
    int i, j, a[2][3];
    printf("Enter elements:\n");
    for(i = 0; i < 2; i++)
        for(j = 0; j < 3; j++)
            scanf("%d", &a[i][j]);
    printf("\nArray elements are:\n");
    for(i = 0; i < 2; i++)
    {
        for(j = 0; j < 3; j++)
            printf("%d ", a[i][j]);
        printf("\n");
    }
    return 0;
}
```

Sample Output :

Enter elements:
1 2 3
4 5 6

Array elements are:
1 2 3
4 5 6

Memory Trick :

2-D Array = Table
`a[i][j]`
(i = row, j = column)

Q2. What are strings in C? Explain different ways to initialize a string. Write the important string handling functions in C with suitable examples. [12 Marks]

Ans :

1. String in C :

- A string is a collection of characters terminated by a null character `'\0'`.
- In C, a string is actually a character array.
- **Example :** `char str[20] = "Hello";`

2. Ways to Initialize a String :

- Using string constant :
`char str1[20] = "Hello";`
- Character by character :
`char str2[20] = {'H', 'e', 'l', 'l', 'o', '\0'};`
- Using pointer :
`char *str3 = "Hello";`
- Taking input at runtime :
`char str4[20];`
`scanf("%s", str4);`

3. Important String Handling Functions :

Function	Purpose	Example
<code>strlen()</code>	Returns length of string	<code>strlen(str)</code>
<code>strcpy()</code>	Copies one string to another	<code>strcpy(s1, s2)</code>
<code>strcat()</code>	Concatenates two strings	<code>strcat(s1, s2)</code>
<code>strcmp()</code>	Compares two strings	<code>strcmp(s1, s2)</code>
<code>strrev()</code>	Reverses a string (non-standard)	<code>strrev(str)</code>
<code>strupr()</code>	Converts to upper case (non-standard)	<code>strupr(str)</code>
<code>strlwr()</code>	Converts to lower case (non-standard)	<code>strlwr(str)</code>

4. Example Program : (String Operations)

```
#include <stdio.h>
#include <string.h>
int main()
{
    char s1[20] = "Hello";
    char s2[20] = "World";
    printf("Length = %d\n", strlen(s1));
    strcat(s1, s2);
    printf("After concatenation = %s\n", s1);
    return 0;
}
```

Sample Output :

Length = 5
After concatenation = HelloWorld

9. Conclusion :

- Strings are character arrays ending with `'\0'`.
- String functions provide easy way to manipulate strings.
- They are widely used in real world programming.

Memory Trick :

String = Characters + `'\0'`
Main Functions: Length, Copy, Concat, Compare

Study
Smart
Write Exact
Score High !

MICRO-CHEATSHEET : UNIT - 3**(Arrays and Strings)****Unit-3 Topics :**

- ✓ Defining & processing arrays
- ✓ Passing arrays to functions
- ✓ Multi-dimensional arrays
- ✓ Arrays and strings

Q1. What is an array? What is a two-dimensional array? How are two-dimensional arrays initialized? Also explain how array elements are accessed in a two-dimensional array. [12 Marks]
Give explanation with suitable example.

Ans :**1. Array :**

- An array is a collection of elements of the same data type stored in contiguous memory locations.
- It is used to store multiple values under a single name.
- **Syntax :** data_type array_name[size];
- **Example :** int a[5];

2. Two-dimensional array :

- A two-dimensional array is an array of arrays. It is represented in the form of rows and columns (matrix).
- **Syntax :** data_type array_name[rows][columns];
- **Example :** int a[2][3]; (2 rows and 3 columns)

3. Initialization of 2-D array :

- A two-dimensional array can be initialized row-wise during declaration.
- **Example :** int a[2][3] = { {1, 2, 3}, {4, 5, 6} };

PYQ ALERT : REPEATED / MOST VVI

Asked in Years : 2015 (Verified)

(Same /related topic may appear again)

Marks : 12 (in 2015 paper - Same question)

4. Accessing elements in 2-D array :

- Each element is accessed using its row index and column index.
- **Syntax :** array_name[i][j]
- **Example :** a[0][0], a[1][2] etc.

Example :

```
#include <stdio.h>
int main()
{
    int a[2][3] = { {1,2,3}, {4,5,6} };
    printf("%d", a[1][2]); // Output = 6
    return 0;
}
```

Memory representation (2x3)

	j=0	j=1	j=2
i=0	1	2	3
i=1	4	5	6

a[1][2] = 6

5. Conclusion :

→ Arrays help to store and process large amount of similar data.

Two-dimensional arrays are useful to represent data in tabular form (matrix).

Memory Trick :

2-D Array = Row x Column

Q2. What is a string in C? Explain how strings are declared, initialized and processed. [12 Marks]
Write a program to perform string operations (such as length, copy and concatenation).

Ans :**1. String :**

- A string is a sequence of characters terminated by a null character "\0".
- In C, a string is treated as a character array.
- **Example :** "Hello" is stored as

H	e	l	l	o	\0
0	1	2	3	4	5

2. Declaration of string :

- char str[size];
- **Example :** char name[20];

3. Initialization of string :

- **During declaration :**
char str[10] = "Hello";
- **Using input functions :**
scanf("%s", str);
- **Using library function :**
gets(str); // (not recommended)
fgets(str, size, stdin);

4. Processing / String operations :

- ① Length of string → strlen() (in <string.h>)
- ② Copy string → strcpy()
- ③ Concatenate string → strcat()
- ④ Compare string → strcmp()

Key Point :

String functions are in <string.h> header file.

Example Program : (Length, Copy and Concatenation)

```
#include <stdio.h>
#include <string.h>
int main()
{
    char s1[20] = "Hello";
    char s2[20];
    char s3[40];

    printf("Length of s1 = %d\n", strlen(s1));

    strcpy(s2, s1);
    printf("After copy s2 = %s\n", s2);

    strcat(s1, " World");
    printf("After concatenation s1 = %s\n", s1);

    return 0;
}
```

Sample Output :

Length of s1 = 5
After copy s2 = Hello
After concatenation
s1 = Hello World

2. Conclusion :

→ Strings are widely used in C for text processing. Library functions make string handling simple and efficient.

Memory Trick :**"SCCL"**

S - strlen (length)
C - strcpy (copy)
C - strcat (concat)
L - strcmp (compare)

Study
Smart
Write Exact
Score High !

MICRO-CHEATSHEET : UNIT - 4**(Structures and Unions - User Defined Data Types & Bitwise Operations)****Unit-4 Topics :**

- ✓ Structures
- ✓ Passing structure to functions
- ✓ Self-referential structures
- ✓ Unions
- ✓ Bitwise operations

Q1. What is a structure? How is a structure defined and initialized in C? Explain accessing structure members and passing structures to functions with example program. [12 Marks]

PYQ ALERT : REPEATED / MOST VVI

Asked in Years : 2015, 2016, 2019, 2021, 2022
(Same / related question).
Marks : 12

Ans :**1. Definition :**

- A structure is a user defined data type that can store different types of data items under a single name.
- It is used to represent a record (e.g., student, employee etc.).

2. Syntax :

```
struct structure_name
{
    data_type member1;
    data_type member2;
    ...
};
```

3. Example (Declaration and Initialization) :

```
#include <stdio.h>
struct student
{
    int roll;
    char name[30];
    float marks;
};
int main()
{
    struct student s1 = {1, "Ali", 78.5};
    printf("Roll: %d\n", s1.roll);
    printf("Name: %s\n", s1.name);
    printf("Marks: %.2f\n", s1.marks);
    return 0;
}
```

Accessing Members :

- Using dot operator (.)
s1.roll
- Using pointer (->)
ptr->roll

Output :

Roll: 1
Name: Ali
Marks: 78.50

4. Passing Structure to Functions :

- A structure can be passed to a function in two ways :
(i) Call by value (whole structure is copied).
(ii) Call by reference (using pointer).

- Example (Call by value) :

```
void display(struct student s)
{
    printf("%d %s %.2f", s.roll, s.name, s.marks);
}
```

5. Conclusion :

- Structures allow us to store heterogeneous data efficiently.
- They can be easily passed to functions and accessed using . and -> operators.

Memory Trick : "STRUCT = Different Data Together"

Q2. What is a union? Explain the difference between structure and union. Also explain bitwise operations in C with suitable examples. [12 Marks]

PYQ ALERT : REPEATED / MOST VVI

Asked in Years : 2014, 2015, 2017, 2021, 2022
(Same / related question).
Marks : 12

Ans :**1. Union :**

- A union is a user defined data type similar to structure, but all members share the same memory location.
- Only one member can store value at a time.

→ Syntax :

```
union union_name
{
    data_type member1;
    data_type member2;
    ...
};
```

Example :

```
union test
{
    int x;
    float y;
    char c;
};
```

2. Structure vs Union :

Feature	Structure	Union
Memory Allocation	Separate memory for each member	Same memory for all members
Size	Sum of sizes of all members	Size of largest member
Access	All members can be used	Only one member at a time
Use	Store multiple values	Save memory
Keyword	struct	union

3. Example to show size difference :

```
#include <stdio.h>
struct demo
{
    int a;
    char b;
    float c;
};
union demo1
{
    int a;
    char b;
    float c;
};
int main()
{
    printf("Structure size: %d\n", sizeof(struct demo));
    printf("Union size: %d\n", sizeof(union demo1));
    return 0;
}
```

Output (on 32 bit) :

Structure size: 9 or 12
Union size: 4
(= size of largest member)

4. Bitwise Operations :

- Bitwise operators work on individual bits of an integer.

Operator	Name	Example (a = 5, b = 3)	Result
&	AND	5 & 3 (0101 & 0011)	1
	OR	5 3 (0101 0011)	7
^	XOR	5 ^ 3 (0101 ^ 0011)	6
~	NOT	~5 (1's complement)	-6
<<	Left Shift	5 << 1 (0101 << 1)	10
>>	Right Shift	5 >> 1 (0101 >> 1)	2

5. Conclusion :

- Union is useful for memory optimization.
- Bitwise operations are used in low-level programming, embedded systems, etc.

Memory Trick : "UNION = One Memory, Many Names | Bitwise = Work on Bits"

- ✓ Study Smart
- ✓ Write Exact
- ✓ Score High !

MICRO-CHEATSHEET : UNIT - 5**(The C Preprocessor)****Unit-5 Topics :**

- ✓ #define to implement constants
- ✓ #define to create functional macros
- ✓ Reading from other files using #include
- ✓ Conditional selection using #ifdef
- ✓ Using #undef
- ✓ Other preprocessor commands
- ✓ Predefined names
- ✓ Macros vs Functions

Q1. What is C Preprocessor? Explain various preprocessor directives with suitable examples. Also explain how to include and read from other files. [12 Marks]

Ans :

1. C Preprocessor :

- The C Preprocessor is a program that processes the source code before compilation.
- It handles directives that start with # (hash).
- It performs tasks like macro substitution, file inclusion, conditional compilation etc.
- Preprocessing is done automatically by the compiler.

2. Common Preprocessor Directives :

Directive	Purpose	Example
#include	Include contents of a file	#include <stdio.h>
#define	Define a constant or macro	#define PI 3.14
#undef	Remove a macro definition	#undef PI
#ifdef	Compile code if macro is defined	#ifdef DEBUG
#ifndef	Compile code if macro is not defined	#ifndef MAX
#if	Conditional compilation	#if x > 0
#else	Alternative block	#else
#endif	End of conditional block	#endif
#pragma	Implementation specific command	#pragma startup
#error	Generate a compilation error	#error "Wrong OS"
#line	Change line number information	#line 100

3. File Inclusion (Reading from other files) :

- We can include another file using #include directive.
- Two types :
 - (i) System header files : Written using angle brackets < >

Example : #include <stdio.h>

- (ii) User defined header files : Written using double quotes ""

Example : #include "myfile.h"

Example :

```
#include <stdio.h>
#include "myfile.h" // user defined file
int main()
{
    printf("Hello from included file!\n");
    return 0;
}
```

Header file (.h) contains function declarations, macros or constants. Its content is copied at preprocessing stage.

4. Conclusion :

- Preprocessor helps to make code modular, reusable and portable.
- #include reads content from other files.
- Preprocessor directives are not part of the C language syntax, but they are handled before compilation.

Memory Trick : "# = Handle Before Compile"

Q2. What are macro? Explain how to create functional macros using #define. Discuss conditional compilation using #ifdef, #ifndef and #undef with examples. [12 Marks]

Ans :

1. Macro :

- A macro is a symbolic name defined using #define.
- The preprocessor replaces the macro name with its value before compilation.
- Macros can be object-like (constants) or function-like.

2. Types of Macros :

Type	Syntax	Example	Use
Object-like Macro	#define name value	#define PI 3.14	Define constant values
Function-like Macro	#define name(param) expression	#define SQR(x) ((x)*(x))	Acts like a function with arguments

3. Example of Functional Macro :

```
#include <stdio.h>
#define SQUARE(x) ((x) * (x))
int main() {
    int a = 5;
    printf("Square = %d\n", SQUARE(a));
    return 0;
}
```

Output :
Square = 25

4. Conditional Compilation :

- Used to include or exclude a block of code based on a condition.
- Useful for debugging and platform specific code.

(i) **#ifdef** : Code is included if the macro is defined.

```
#define DEBUG
#ifdef DEBUG
    printf("Debug mode enabled\n");
#endif
```

(ii) **#ifndef** : Code is included if the macro is not defined.

```
#ifndef MAX
#define MAX 100
#endif
```

(iii) **#undef** : Used to remove a macro definition.

```
#define PI 3.14
#undef PI
```

Example :

```
#include <stdio.h>
#define TEST
int main() {
    #ifdef TEST
        printf("Test On\n");
    #else
        printf("Test Off\n");
    #endif
    #undef TEST
    return 0;
}
```

5. Conclusion :

- Macros reduce code size and make programs flexible.
- Functional macros work like inline functions.
- Conditional compilation helps in debugging and making code portable.
- #undef removes a macro definition.

Memory Trick : "IFDEF = If Defined, IFNDEF = If Not Defined, UNDEF = UnDefine"

MICRO-CHEATSHEET : UNIT - 5

(The C Preprocessor)

Exact Exam Oriented Notes | AKU Patna | Write in Exam Copy | 12 Marks Ready

Unit-5 Topics :

- ✓ Define & implement constants
- ✓ Create functional macros
- ✓ Read from other files
- ✓ #include directive
- ✓ Conditional compilation (#ifdef)
- ✓ Predefined macros
- ✓ Macro vs Function

Q1. What is C Preprocessor? Define and implement constants. Explain different ways to create functional macros with examples. [12 Marks]

PYQ ALERT :

Asked in Years : 2014, 2015, 2016, 2017, 2019, 2021, 2022
(Same / related question).
Marks : 12

1. C Preprocessor :

- Preprocessor is a program that processes the source code before compilation.
- It handles directives starting with # (hash).
- It performs tasks like file inclusion, macro expansion, constant definition, and conditional compilation.

2. Constants using #define :

- #define directive is used to create symbolic constants.
- The value is replaced by preprocessor before compilation.

Example :

```
#include <stdio.h>
#define PI 3.1416
#define MAX 100
int main()
{
    printf("PI = %f\n", PI);
    printf("MAX = %d\n", MAX);
    return 0;
}
```

Output :

```
PI = 3.1416
MAX = 100
```

Note :

Use #define for constants. No memory is allocated.

3. Functional Macros :

- A macro that takes arguments and performs a function-like task is called functional macro.
- **Syntax :** #define macro_name(arguments) expression
- Different ways to create functional macros :

(i) Single argument macro :

```
#define SQUARE(x) ((x)*(x))
```

(ii) Multi-argument macro :

```
#define MAX(a,b) ((a) > (b) ? (a) : (b))
```

(iii) Macro with expression :

```
#define AREA(r) (3.1416 * (r) * (r))
```

(iv) Macro with multiple statements (using do...while) :

```
#define PRINT(x) \
do { printf("Value = %d\n", (x)); } while(0)
```

4. Example Program :

```
#include <stdio.h>
#define ADD(a,b) ((a)+(b))
#define MUL(a,b) ((a)*(b))
int main()
{
    int x=5, y=3;
    printf("Sum = %d\n", ADD(x,y));
    printf("Product = %d\n", MUL(x,y));
    return 0;
}
```

Output :

```
Sum = 8
Product = 15
```

Q2. Explain #include directive. How can we read from other files? Explain conditional compilation using #ifdef and #ifndef. Mention some predefined macros and their uses. [12 Marks]

PYQ ALERT :

Asked in Years : 2014, 2015, 2016, 2019, 2021, 2022
(Same / related question).
Marks : 12

1. #include Directive :

- Used to include the content of another file into current file.
- **Syntax :** #include <filename> (for system files)
#include "filename" (for user defined files)
- **Example :**

```
#include <stdio.h> // system file (standard library)
#include "myfile.h" // user defined file
```

2. Reading from Other Files :

- We can read (include) another file using #include directive.
- The included file may contain function definitions, header files or macro definitions.
- **Example :**

myfile.h

```
#define MSG "Hello"
void show();
```

main.c

```
#include <stdio.h>
#include "myfile.h"
void show()
{
    printf("%s\n", MSG);
}
```

3. Conditional Compilation :

- Used to include or exclude a part of code based on conditions.
- Common directives : #ifdef, #ifndef, #else, #elif, #endif

Example :

```
#define DEBUG
#ifdef DEBUG
    printf("Debug mode\n");
#else
    printf("Release mode\n");
#endif
```

Note :

If DEBUG is defined, only first part will be compiled.

4. Predefined Macros :

Macro	Meaning	Example Use
__FILE__	Name of current file	printf("%s", __FILE__);
__LINE__	Current line number	printf("%d", __LINE__);
__DATE__	Compilation date	printf("%s", __DATE__);
__TIME__	Compilation time	printf("%s", __TIME__);
__STDC__	Standard C (1 if true)	#if __STDC__
__cplusplus	Defined in C++	#ifdef __cplusplus

5. Conclusion :

- Preprocessor makes the program flexible, modular and easy to maintain.
- It helps in constant definition, file inclusion, macro expansion and conditional compilation.

Memory Trick : "P I M C P"

P = Preprocessor I = Include M = Macro
C = Conditional P = Predefined

Use #include for files, #define for constants/macros, #ifdef for conditions.

Study
Smart
Write Exact
Score High !

MICRO - CHEATSHEET : UNIT - 5

File Handling in C

Unit-5 Topics :

- ✓ File handling in C using file pointers
- ✓ Open a file using fopen()
- ✓ Close a file using fclose()
- ✓ Input / Output using file pointers
- ✓ Character, String, Formatted I/O
- ✓ Block I/O (fread / fwrite)
- ✓ Sequential vs Random access
- ✓ Positioning the file pointer (fseek, ftell, rewind)

Q1. How we open and close an existing file in C?
Also discuss the various modes to open a file.

[12 Marks]

Ans :

1. Opening a File :

- fopen() function is used to open a file.
- **Syntax** : FILE *fp = fopen("filename", "mode");
- If file opens successfully, it returns a file pointer.
- If it fails, it returns NULL.

2. Closing a File :

- fclose() function is used to close a file.
- **Syntax** : fclose(fp);
- It flushes the buffer and releases the file.

3. File Open Modes :

Mode	Meaning	Use case
r	Open for reading (file must exist)	Read data
w	Open for writing (creates new / truncates existing)	Write data (old data lost)
a	Open for appending (creates if not exist)	Add data at end
r+	Open for reading and writing (must exist)	Read & write
w+	Open for reading and writing (create / truncate)	Read & write (new file)
a+	Open for reading and appending	Read & append

4. Example Program : (Open, read and close a file)

```
#include <stdio.h>
int main()
{
    FILE *fp;
    char ch;
    fp = fopen("example.txt", "r");
    if(fp == NULL)
    {
        printf("File cannot be opened\n");
        return 0;
    }
    while((ch = fgetc(fp)) != EOF)
        printf("%c", ch);
    fclose(fp);
    return 0;
}
```

Sample Output :

(Contents of
example.txt)Hello C
File Handling

5. Conclusion :

- fopen() is used to open a file, fclose() is used to close it.
- Different modes are used depending on whether we want to read, write, append or both.
- Always check for NULL while opening a file.

Memory Trick : "R = Read, W = Write, A = Add
+ means both (R+ / W+ / A+)"

Q2. Explain various input and output functions in file handling (character, string, formatted and block I/O) with examples. [12 Marks]

Ans :

1. Character I/O :

- fgetc() - reads a character from a file.
- fputc() - writes a character to a file.

```
FILE *fp = fopen("test.txt", "r");
char ch = fgetc(fp);

FILE *fp = fopen("test.txt", "w");
fputc('A', fp);
```

Functions :

fgetc(fp)
fputc(ch, fp)

2. String I/O :

- fgets() - reads a string (line) from a file.
- fputs() - writes a string to a file.

```
char str[100];
FILE *fp = fopen("test.txt", "r");
fgets(str, 100, fp);

FILE *fp = fopen("test.txt", "w");
fputs("Hello C\n", fp);
```

Functions :

fgets(str, n, fp)
fputs(str, fp)

3. Formatted I/O :

- fscanf() - reads formatted data from a file.
- fprintf() - writes formatted data to a file.

```
int roll;
char name[20];
FILE *fp = fopen("data.txt", "r");
fscanf(fp, "%d %s", &roll, name);

fprintf(fp, "%d %s", roll, name);
```

Functions :

fscanf(fp, format, ...)
fprintf(fp, format, ...)

4. Block I/O :

- fread() - reads a block of data (binary).
- fwrite() - writes a block of data (binary).

```
int n;
struct student s;
FILE *fp = fopen("stud.dat", "rb");
fread(&s, sizeof(s), 1, fp);

fp = fopen("stud.dat", "wb");
fwrite(&s, sizeof(s), 1, fp);
```

Functions :

fread(ptr, size, n, fp)
fwrite(ptr, size, n, fp)

5. Conclusion :

- We can perform character, string, formatted and block I/O using different functions in C.
- These functions make file handling flexible and powerful.

Memory Trick : "GCSFB = Get, Char, String, Formatted, Block"



Smart Study
Bright Future

arrahmanstudyhub.tech

Keep Learning !
You Can Do It !

Study
Smart
Write Exact
Score High !

MICRO-CHEATSHEET : UNIT - 5**(The C Preprocessor)**

Short Notes + Exact Answers + Exam Focused

Unit-5 Topics :

- #define to implement constants
- #define to create functional macros
- Reading from other files using #include
- Conditional selection using #ifdef
- Using #ifdef for different computer types
- Other preprocessor commands
- Predefined names, macros

Q1. Explain the role of C preprocessor. Discuss various preprocessor directives with examples.

[12 Marks]

Ans :**1. Introduction :**

- The C preprocessor is a program that processes the source code before compilation.
- It handles directives that start with # (hash).
- It performs text substitution, file inclusion, conditional compilation and macro expansion.
- It does not check for syntax errors. Its output is a modified source code which is then given to the compiler.

2. Common Preprocessor Directives :

Directive	Purpose	Example
#include	Include header file	#include <stdio.h>
#define	Define constant or macro	#define PI 3.14
#undef	Undefine a macro	#undef PI
#ifdef	Compile code if macro defined	#ifdef DEBUG ... #endif
#ifndef	Compile code if macro not defined	#ifndef MAX ... #endif
#if	Conditional compilation	#if 0 ... #endif
#else	Alternative block	#if ... #else ... #endif
#elif	Else if condition	#elif defined(LINUX)
#error	Generate error message	#error "Wrong Compiler"
#pragma	Implementation specific	#pragma warning
defined()	Check if macro is defined	#if defined(WIN32)

3. Example Program (Constants + File Inclusion + Conditional) :

```
#include <stdio.h> // header file
#define MAX 100 // constant

int main()
{
    #ifdef MAX
        printf("MAX = %d\n", MAX);
    #else
        printf("MAX is not defined\n");
    #endif
    return 0;
}
```

Output :
MAX = 100

Preprocessor runs first,
Compiler runs after that.

4. Conclusion :

- Preprocessor makes the program flexible, portable and easy to maintain.
- It helps in code reuse and conditional compilation.

Memory Trick : "# Preprocess Before Compile"

Q2. Explain the difference between #define and const. Also explain how to create and use functional macros.

[12 Marks]

Ans :**1. #define vs const :**

Feature	#define	const
Type	Preprocessor directive	Language keyword
Processing	Text substitution	Handled by compiler
Data type	No type checking	Type checking done
Memory	No memory allocated	Memory allocated
Scope	Global (till #undef)	Respects scope rules
Debugging	Difficult	Easier

2. Example :

```
#define PI 3.14
int r = 5;
float area = PI * r * r;
```

```
const float PI = 3.14;
int r = 5;
float area = PI * r * r;
```

Use **const** for type safety. Use **#define** for simple substitution.

3. Functional Macros :

- A macro with parameters is called functional macro.
- It is defined using #define with arguments.
- It simply replaces the macro call with the given expression.

Example :

```
#define SQUARE(x) ((x)*(x))
#include <stdio.h>
int main()
{
    int a = 5;
    printf("Square = %d\n", SQUARE(a));
    return 0;
}
```

Output :
Square = 25

Functional macros do not check the type of arguments.

Q3. How do we include header files? Explain with example. Also discuss using #ifdef for different computer types.

[12 Marks]

Ans :**1. Including Header Files :**

- The #include directive is used to include header files.
- Two types :
 - (i) Standard header files (in <>) → e.g. stdio.h
 - (ii) User defined header files (in " ") → e.g. "myfile.h"

Example :

```
#include <stdio.h> // standard
#include "myfile.h" // user defined
```

<> → search in system directory
" " → search in current directory

2. Using #ifdef for Different Computer Types :

- The #ifdef directive is used to compile code for specific computer or OS.
- Predefined macros like __WIN32, __linux__, __APPLE__ can be used.

Example :

```
#ifdef __WIN32
    printf("Windows OS\n");
#elif defined(__linux__)
    printf("Linux OS\n");
#else
    printf("Other OS\n");
#endif
```

Helps in writing portable programs for different platforms.

3. Conclusion :

- Header files provide library functions and macros.
- #ifdef helps in conditional compilation for different computer types.

Memory Trick : "Include = <> or " " | #ifdef = If Defined"

All the Best for Exam !
😊

Study
Smart
Write Exact
Score High !

MICRO-CHEATSHEET : UNIT - 5

(The C Preprocessor)

Unit-5 Topics :

- ✓ #define to implement constants
- ✓ #define to create functional macros
- ✓ Reading from other files using #include
- ✓ Conditional selection of code using #ifdef
- ✓ Using #ifndef for different computer types
- ✓ Other preprocessor commands
- ✓ Predefined names
- ✓ Macros vs Functions

Q1. What is C Preprocessor? Explain #define to implement constants and to create functional macros with examples.

[12 Marks]

Ans :

1. C Preprocessor :

- The C Preprocessor is a program that processes the source code before compilation.
- It handles all preprocessor directives (lines starting with #).
- It performs tasks like macro substitution, file inclusion, conditional compilation etc.
- The output of preprocessor is called "expanded source code", which is then compiled by the compiler.

2. #define to implement constants :

- #define is used to create symbolic constants.
- The preprocessor replaces the symbol with its value before compilation.

Example :

```
#include <stdio.h>
#define PI 3.1416
#define MAX 100
int main()
{
    printf("PI = %f\n", PI);
    printf("MAX = %d\n", MAX);
    return 0;
}
```

Output :

```
PI = 3.1416
MAX = 100
```

Note :

No memory is allocated for define constants. It is just a text replacement.

3. Functional Macros :

- A macro that accepts parameters is called a function-like macro.
- It is expanded inline by the preprocessor.

Example :

```
#include <stdio.h>
#define SQUARE(x) ((x)*(x))
#define MAX(a,b) ((a)>(b)?(a):(b))
int main()
{
    int a = 5, b = 8;
    printf("Square of a = %d\n", SQUARE(a));
    printf("Maximum = %d\n", MAX(a,b));
    return 0;
}
```

Output :

```
Square of a = 25
Maximum = 8
```

Memory Trick :

Macro = Copy-Paste by Preprocessor)

4. Advantages of Macros :

- Faster execution (no function call overhead).
- Useful for small and frequently used code.
- Can work with different data types.
- Used for constants and conditional compilation.

5. Limitations :

- No type checking.
- Difficult to debug.
- Can lead to unexpected results if not used carefully (e.g. missing parentheses).

6. Conclusion :

- #define is used to create symbolic constants and function-like macros.
- It improves code readability and maintainability.
- Use parentheses in macros to avoid errors.

Memory Trick : "#define = Before Compile, Replace and Shine"

Q2. Explain #include, conditional compilation using #ifdef, other preprocessor commands and predefined names.

[12 Marks]

Ans :

1. #include Directive :

- Used to include the contents of another file in the program.
- Two types :
 - (i) <filename> : Standard header files (search in system directory).
 - (ii) "filename" : User defined header files (search in current directory).

Example :

```
#include <stdio.h> // Standard header file
#include "myheader.h" // User defined header file
```

Common Header Files :

Header File	Purpose
stdio.h	Input / Output
stdlib.h	Standard library
math.h	Mathematical functions
string.h	String handling
conio.h	Console functions (Turbo C)

2. Conditional Compilation using #ifdef :

- Used to include or exclude a block of code based on whether a symbol is defined or not.
- Useful for debugging and for different systems.

Example :

```
#define DEBUG
#include <stdio.h>
int main()
{
    printf("Hello\n");
    #ifdef DEBUG
        printf("Debug mode ON\n");
    #else
        printf("Debug mode OFF\n");
    #endif
    return 0;
}
```

If DEBUG is defined, 'Debug mode ON' will be printed.

Other Conditional Directives :

#ifndef	true if not defined
#if	conditional expression
#else	alternative block
#elif	else if condition
#endif	end of conditional block

3. Other Preprocessor Commands :

Directive	Purpose
#undef	Undefines a macro
#pragma	Provides special instructions to compiler
#line	Changes the current line number
#error	Generates an error message
#warning	Generates a warning message

4. Predefined Names (Macros) :

Macro	Meaning
__FILE__	Current file name
__LINE__	Current line number
__DATE__	Compilation date
__TIME__	Compilation time
__STDC__	1 if standard C compiler

Example :

```
#include <stdio.h>
int main()
{
    printf("File : %s\n", __FILE__);
    printf("Line : %d\n", __LINE__);
    printf("Date : %s\n", __DATE__);
    printf("Time : %s\n", __TIME__);
    return 0;
}
```

These are automatically defined by the compiler.

5. Using #ifdef for Different Computer Types :

- Used to write system dependent code.

Example :

```
#ifdef _WIN32
    printf("Windows System\n");
#elif __linux__
    printf("Linux System\n");
#else
    printf("Unknown System\n");
#endif
```

6. Conclusion :

- Preprocessor simplifies program development.
- #include, #define and #ifdef are most useful directives.
- Predefined macros help in debugging and portability.

Memory Trick : "IDEF → Include, Define, Evaluate, Freedom"

Q1. What is a File? Explain file handling in C with file pointer and basic functions. [12 Marks]

Ans :

1. File :

- A file is a named location on disk used to store data permanently.
- It can store text data or binary data.
- File handling allows us to create, read, write, update and close files using library functions in C.

2. File Pointer :

- A file pointer is a pointer of type **FILE** (defined in <stdio.h>).
- It points to a file and is used to perform file operations.

Declaration : FILE *fp;

3. Steps to Work with Files :

1. Declare a file pointer.
2. Open the file using fopen().
3. Perform read/write operations.
4. Close the file using fclose().
5. Always check for errors (if fp == NULL).

File =
Permanent
Storage

4. fopen() - Syntax & Modes :

FILE *fopen(const char *filename, const char *mode);

Mode	Description	Use
r	Open for reading (file must exist)	Read data
w	Open for writing (create/new or truncate)	Write data
a	Open for appending (create if not exist)	Append data
r+	Read and write (file must exist)	Both
w+	Read and write (create/new or truncate)	Both
a+	Read and append (create if not exist)	Both

5. Common File Functions :

Function	Purpose
fopen()	Open a file
fclose()	Close a file
fprintf()	Write formatted data (text)
fscanf()	Read formatted data (text)
fgetc()	Read a single character
fputc()	Write a single character
fgets()	Read a string (line)
fputs()	Write a string
fread()	Read binary data
fwrite()	Write binary data
feof()	Check end of file
ferror()	Check error in file
rewind()	Move file pointer to beginning
fseek()	Move file pointer to a specific position
ftell()	Get current position of file pointer
remove()	Delete a file
rename()	Rename a file

Header File
<stdio.h>
(For File Handling)

6. Error Checking :

```
fp = fopen("data.txt", "r");
if(fp == NULL) {
    printf("File cannot be opened!\n");
    return 0;
}
```

Q2. Write a program to create a file, write data into it and read the data from the file. [12 Marks]

Ans :

```
#include <stdio.h>
int main() {
    FILE *fp;
    char name[50];
    int age;

    // Open file in write mode
    fp = fopen("student.txt", "w");
    if(fp == NULL) {
        printf("File cannot be created!\n");
        return 0;
    }

    printf("Enter name: ");
    gets(name);
    printf("Enter age: ");
    scanf("%d", &age);

    // Write data to file
    fprintf(fp, "Name: %s\nAge: %d\n", name, age);
    fclose(fp);

    // Open file in read mode
    fp = fopen("student.txt", "r");
    if(fp == NULL) {
        printf("File cannot be opened!\n");
        return 0;
    }

    printf("\nData from file:\n");
    while((fgets(name, 50, fp)) != NULL) {
        printf("%s", name);
    }

    fclose(fp);
    return 0;
}
```

Sample Input :

Enter name: Zoha
Enter age: 20

Sample Output :

Data from file:
Name: Zoha
Age: 20

Important Points :

- ✓ Always check if file opened successfully.
- ✓ Use fprintf()/fscanf() for formatted I/O.
- ✓ Use fgets()/fputs() for string I/O.
- ✓ Close the file after use (fclose()).

Other Important Programs (Likely in Exam) :

1. Write a program to copy contents of one file to another file.
2. Write a program to count number of characters, words and lines in a file.
3. Write a program to append data to an existing file.
4. Write a program to delete a file using remove().
5. Write a program to rename a file using rename().
6. Write a program to read and display a file using fgetc().
7. Write a program to store structure data in a file (using fwrite() and fread()).

Key Points to Remember :

- ✓ File handling is not part of C language syntax, it is handled by library functions.
- ✓ Data in a file is permanent (even after program ends).
- ✓ Use binary mode (rb, wb, ab) for binary data like structure.
- ✓ Use text mode for normal data.
- ✓ Always close the file using fclose().
- ✓ Use feof() to check end of file.
- ✓ fseek(), ftell() and rewind() are useful for random access.
- ✓ Use remove() to delete a file and rename() to rename a file.

File
Handling
= More Control
+
Permanent
Data

Conclusion :

- File handling allows us to store and retrieve data permanently.
- C provides many library functions for file operations.
- It is useful for real-world applications like databases, logs, and record keeping.

Memory Trick :

"Open → Operate → Close" (fopen → read/write → fclose)



MICRO-CHEATSHEET : UNIT - 6

"Files make data
Permanent !
Learn once,
use everywhere !"



(FILES)



Unit-6 Topics :

- ✓ File Handling in C using File Pointers
- ✓ Open a file using fopen()
- ✓ Close a file using fclose()
- ✓ Input and Output using file pointers
- ✓ Character Input and Output in files
- ✓ String Input / Output Functions
- ✓ Formatted Input / Output Functions
- ✓ Block Input / Output Functions
- ✓ Sequential vs Random Access Files
- ✓ Positioning the File Pointer

Q1. Explain file handling in C using file pointers. How to open a file using fopen(), perform read/write operations (using fprintf(), fscanf(), fgetc(), fputc(), fgets(), fputs() etc.) and close a file using fclose().

Ans :

[12 Marks]

1. What is a File ?

- A file is a collection of data stored on secondary storage (permanently).
- In C, file handling is done using file pointers and functions from <stdio.h>.

2. File Pointer

- A file pointer is a pointer of type FILE used to point to a file.
- Syntax: `FILE *fp;`

3. Opening a File - fopen()

- Syntax: `fp = fopen("filename", "mode");`
- Modes :

Mode	Description	File Action
r	Open for reading	File must exist
w	Open for writing	Creates new (or overwrites)
a	Open for appending	Adds data at end
r+	Read and write	File must exist
w+	Read and write	Creates new (or overwrites)
a+	Read and append	Adds data at end

4. Writing to a File

- fprintf() → formatted output
- fputc() → single character
- fputs() → string output

Example :

```
FILE *fp;
fp = fopen("data.txt", "w");
if(fp == NULL) {
    printf("File cannot be opened");
    return 0;
}
fprintf(fp, "Hello %d", 100);
fputc('A', fp);
fputs("\nC Programming", fp);
fclose(fp);
```

Write data
into file

5. Reading from a File

- fscanf() → formatted input
- fgetc() → single character
- fgets() → string input

Example :

```
FILE *fp;
char name[50];
int age;
fp = fopen("data.txt", "r");
if(fp == NULL) {
    printf("File cannot be opened");
    return 0;
}
fscanf(fp, "%s %d", name, &age);
fgets(name, 50, fp);
char ch = fgetc(fp);
fclose(fp);
```

Read data
from file

Memory Trick : "r = Read, w = Write, a = Add (Append)"

6. Closing a File - fclose()

- Syntax: `fclose(fp);`
- It closes the file and flushes the buffers.

7. File I/O Functions Overview

Function	Purpose	Example
fopen()	Open a file	<code>fp = fopen("test.txt", "r");</code>
fclose()	Close a file	<code>fclose(fp);</code>
fprintf()	Write formatted data	<code>fprintf(fp, "%d", n);</code>
fscanf()	Read formatted data	<code>fscanf(fp, "%d", &n);</code>
fputc()	Write a character	<code>fputc('A', fp);</code>
fgetc()	Read a character	<code>ch = fgetc(fp);</code>
fputs()	Write a string	<code>fputs("Hello", fp);</code>
fgets()	Read a string	<code>fgets(str, 50, fp);</code>

8. Sequential vs Random Access File

Sequential Access File	Random Access File
▶ Data is read/written in sequence. ▶ File pointer moves automatically one by one. ▶ Used for text files. ▶ Functions: fgetc(), fgets(), fscanf(), fprintf(). ▶ Example: Reading a file line by line.	▶ Data can be accessed directly at any position. ▶ Uses file pointer positioning functions. ▶ Used for binary files. ▶ Functions: fseek(), ftell(), rewind(). ▶ Example: Accessing a specific record in a file.

9. File Pointer Positioning Functions

Function	Purpose	Example
fseek()	Move file pointer to a specific position	<code>fseek(fp, 0, SEEK_SET)</code>
ftell()	Get current position of file pointer	<code>long pos = ftell(fp);</code>
rewind()	Move to beginning of file	<code>rewind(fp);</code>

Example : (Using fseek & ftell)

```
FILE *fp;
fp = fopen("data.txt", "r");
fseek(fp, 10, SEEK_SET); // move to 10th byte
long pos = ftell(fp); // get current position
printf("Current position : %ld", pos);
fclose(fp);
```

SEEK_SET → Beginning
SEEK_CUR → Current
SEEK_END → End

10. Conclusion

- File handling allows permanent storage of data.
- Using file pointers and standard functions, we can easily perform read, write, append operations.
- Sequential and random access both are useful in different situations.
- Proper file handling makes programs more flexible and reliable.

Memory Trick : "fopen to open, fclose to close, fprintf to write, fscanf to read"

- ✓ Study Smart
- ✓ Write Exact
- ✓ Score High
- ✓ AKU Exams

MICRO-CHEATSHEET : UNIT - 6

(Files in C Programming)

Small Notes → Big Marks → Sure Success !

Q1. Explain file handling in C using file pointers. How to open and close a file? Discuss various file modes with suitable examples. [12 Marks]

Ans :

1. File Handling in C :

- A file is a collection of data stored on secondary storage.
- In C, file handling is done using file pointers of type FILE.
- We use standard library functions like fopen(), fclose(), fgetc(), fputc(), fprintf(), fscanf() etc.
- File handling helps to store data permanently.

2. File Pointer :

- A file pointer is a pointer of type FILE which points to a file.
- It is declared as : FILE *fp;
- It is used to perform different file operations.
- FILE is defined in <stdio.h> header file.

3. Opening a file :

- The fopen() function is used to open a file.
- Syntax : FILE *fopen(const char *filename, const char *mode);
- It returns a pointer to the file, or NULL if the file cannot be opened.

Example :

```
#include <stdio.h>
int main() {
    FILE *fp;
    fp = fopen("data.txt", "r"); // open for reading
    if(fp == NULL) {
        printf("File cannot be opened.\n");
        return 1;
    }
    printf("File opened successfully!\n");
    fclose(fp); // close the file
    return 0;
}
```

Always
check if
fp == NULL
after
fopen().

4. Closing a file :

- The fclose() function is used to close an open file.
- Syntax : int fclose(FILE *fp);
- It returns 0 on success and EOF on failure.
- Always close the file after completion of operations to free resources.

5. File Modes :

Mode	Meaning	Description
r	Read	Open an existing file for reading. (File must exist)
w	Write	Create a new file for writing. (If exists, old data deleted)
a	Append	Open an existing file for appending.
r+	Read/Write	Open existing file for both reading and writing.
w+	Write/Read	Create a new file for both reading and writing.
a+	Append/Read	Open existing file for both appending and reading.

Example :

```
fopen("test.txt", "r"); // open for reading
fopen("test.txt", "w"); // open for writing
fopen("test.txt", "a"); // open for appending
fopen("test.txt", "r+"); // open for read and write
fopen("test.txt", "w+"); // open for write and read
fopen("test.txt", "a+"); // open for append and read
```

Memory Trick : r = Read, w = Write (new), a = Add (append)

Q2. Explain different file input and output functions in C with suitable examples. [12 Marks]

Ans :

1. Character Input and Output Functions :

- fgetc() - reads a single character from a file.
- fputc() - writes a single character to a file.

Example :

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("char.txt", "w");
    fputc('A', fp); // write a character
    fclose(fp);
    fp = fopen("char.txt", "r");
    char ch = fgetc(fp); // read a character
    printf("Character read: %c\n", ch);
    fclose(fp);
    return 0;
}
```

Uses :

- Reading/Writing single characters
- Useful for character-by-character processing

2. String Input and Output Functions :

- fgets() - reads a string from a file.
- fputs() - writes a string to a file.

Example :

```
FILE *fp = fopen("text.txt", "w");
fputs("Hello C Programming!\n", fp); // write string
fclose(fp);
fp = fopen("text.txt", "r");
char str[100];
fgets(str, 100, fp); // read string
printf("String read: %s", str);
fclose(fp);
```

- fgets() reads complete line (including spaces)
- fputs() writes string without formatting.

3. Formatted Input and Output Functions :

- fprintf() - writes formatted data to a file (like printf).
- fscanf() - reads formatted data from a file (like scanf).

Example :

```
FILE *fp = fopen("num.txt", "w");
int a = 10; float b = 3.14;
fprintf(fp, "%d %2f", a, b); // write formatted data
fclose(fp);
fp = fopen("num.txt", "r");
int x; float y;
fscanf(fp, "%d %f", &x, &y); // read formatted data
printf("x = %d, y = %2f\n", x, y);
fclose(fp);
```

- fprintf() and fscanf() work like printf() and scanf() but with files.

4. Block Input and Output Functions :

- fread() - reads a block of data from a file.
- fwrite() - writes a block of data to a file.

Example :

```
int arr[5] = {1,2,3,4,5};
FILE *fp = fopen("data.bin", "wb");
fwrite(arr, sizeof(int), 5, fp); // write 5 integers
fclose(fp);

int brr[5];
fp = fopen("data.bin", "rb");
fread(brr, sizeof(int), 5, fp); // read 5 integers
for(int i=0; i<5; i++)
    printf("%d ", brr[i]);
fclose(fp);
```

- fread() and fwrite() are used for binary files and large data like structures, arrays etc.

5. Sequential vs Random Access :

- **Sequential Access :** Data is read/written in order from beginning to end.
- **Random Access :** We can move to any position using fseek(), ftell() and rewind().

6. Positioning the File Pointer :

- fseek(fp, offset, origin); // move file pointer
- ftell(fp); // returns current position
- rewind(fp); // moves pointer to beginning of file

Conclusion : File handling makes data permanent and allows flexible data processing using various I/O functions with file pointers.

Memory Trick : fgetc = Get Character, fputc = Put Character
fgets = Get String, fputs = Put String



MICRO - CHEATSHEET : UNIT - 6

- ✓ Exam Focused
- ✓ Easy Language
- ✓ Exact Answers
- ✓ Score High !

Files in C (File Handling)

1. What is a File ?

- A file is a named location on disk used to store data permanently.
- In C, files are accessed through file pointers using standard library functions.
- File handling allows us to create, open, read, write, update and close files.

2. File Pointer

- A file pointer is a pointer of type FILE.
- It is used to store the address of a file in memory.
- It is declared in <stdio.h> header file.

Syntax : FILE *fp;

3. Opening a File (fopen())

- fopen() is used to open a file and returns a pointer to the file.
- If the file does not exist, it can be created (depending on mode).

Syntax : fp = fopen("filename", "mode");

File Opening Modes

Mode	Meaning	If file exists	If file does not exist
r	Read (text)	Opens for reading	Error
w	Write (text)	Deletes old content	Creates new file
a	Append (text)	Adds at end	Creates new file
r+	Read & Write	Opens for reading and writing	Error
w+	Write & Read	Deletes old content	Creates new file
a+	Append & Read	Adds at end	Creates new file

4. Closing a File (fclose())

- fclose() is used to close an opened file.
- It releases the resources and makes the file not accessible.

Syntax : fclose(fp);

5. Reading and Writing Characters

Function	Purpose
fgetc(fp)	Reads a single character from file.
fputc(ch, fp)	Writes a single character to file.

Example : Write and read a character

Write to file

```
#include <stdio.h>
int main() {
    FILE *fp;
    fp = fopen("test.txt", "w");
    fputc('A', fp);
    fclose(fp);
    return 0;
}
```

Read from file

```
#include <stdio.h>
int main() {
    FILE *fp;
    char ch;
    fp = fopen("test.txt", "r");
    ch = fgetc(fp);
    printf("%c", ch);
    fclose(fp);
    return 0;
}
```

Output
A

Memory Trick : R = Read, W = Write, A = Add | + = Both (Read + Write)

6. Reading and Writing Strings

Function	Purpose
fgets(str, n, fp)	Reads a string (n-1 characters).
fputs(str, fp)	Writes a string to file.

Write a string

```
#include <stdio.h>
int main() {
    FILE *fp;
    char str[100];
    fp = fopen("test.txt", "w");
    printf("Enter a string: ");
    fgets(str, 100, stdin);
    fputs(str, fp);
    fclose(fp);
    return 0;
}
```

Read a string

```
#include <stdio.h>
int main() {
    FILE *fp;
    char str[100];
    fp = fopen("test.txt", "r");
    if(fp != NULL){
        fgets(str, 100, fp);
        printf("%s", str);
        fclose(fp);
    }
    return 0;
}
```

7. Formatted Input / Output

Function	Purpose
fprintf(fp, "format", ...)	Writes formatted data to file.
fscanf(fp, "format", ...)	Reads formatted data from file.

Example : Write and read integers

Write integers

```
#include <stdio.h>
int main() {
    FILE *fp;
    int n;
    fp = fopen("num.txt", "w");
    for(int i=1; i<=5; i++){
        printf("Enter number: ");
        scanf("%d", &n);
        fprintf(fp, "%d\n", n);
    }
    fclose(fp);
    return 0;
}
```

Read integers

```
#include <stdio.h>
int main() {
    FILE *fp;
    int n;
    fp = fopen("num.txt", "r");
    printf("Numbers in file:\n");
    while(fscanf(fp, "%d", &n) != EOF){
        printf("%d\n", n);
    }
    fclose(fp);
    return 0;
}
```

Output
1 2 3 4 5

8. Other Useful Functions

Function	Purpose
feof(fp)	Checks end of file (EOF).
ferror(fp)	Checks if an error occurred.
rewind(fp)	Moves file pointer to beginning.
remove("file")	Deletes a file.
rename("old", "new")	Renames a file.
ftell(fp)	Returns current file position.
fseek(fp, offset, origin)	Moves file pointer (SEEK_SET, SEEK_CUR, SEEK_END).

9. Important Points for Exam

- Always check if file is opened successfully (fp != NULL).
- Use appropriate mode while opening a file.
- Close the file using fclose() after operations.
- Use fprintf() / fscanf() for formatted data.
- Use feof() in a loop to read till end of file.
- File handling makes data permanent (even after program ends).



UNIT 7 – MOST REPEATED QUESTIONS (AKU PYQ)

S. No.	Question	Year Asked
1	How to open and close a file in C? Discuss the various modes to open a file.	2015, 2016, 2018, 2020, 2022, 2024
2	Write a program to copy contents of one file to another file.	2016, 2019, 2021, 2023
3	Write a program to count number of characters / words / lines in a file.	2017, 2020, 2022, 2024
4	Explain different file handling functions in C.	2018, 2021
5	Write a program to merge two files.	2019, 2023

KEY POINTS (QUICK REVISION)

- File = collection of data stored on secondary storage.
- File operations: create, open, read, write, append, close.
- `FILE *fp;` → file pointer declaration.
- Opening a file: `fopen( )`
- Closing a file: `fclose( )`
- Common functions: `fscanf( )`, `fprintf( )`, `fgetc( )`, `fputc( )`, `fgets( )`, `fputs( )`, `feof( )`, `rewind( )`, `fseek( )`, `ftell( )`.
- File modes: `r`, `w`, `a`, `r+`, `w+`, `a+` (text mode)
`rb`, `wb`, `ab`, `rb+`, `wb+`, `ab+` (binary mode)
- Header file: `#include <stdio.h>`

FILE OPENING MODES (MOST ASKED)

Mode	Purpose
<code>r</code>	Open for reading (file must exist)
<code>w</code>	Open for writing (creates new file / overwrites)
<code>a</code>	Open for appending (adds at end)
<code>r+</code>	Open for both reading and writing
<code>w+</code>	Open for both reading and writing (new file / overwrite)
<code>a+</code>	Open for both reading and appending
<code>rb</code> , <code>wb</code> , <code>ab</code>	Binary mode (read / write / append)
<code>rb+</code> , <code>wb+</code> , <code>ab+</code>	Binary mode (read & write)

IMPORTANT FILE FUNCTIONS

Function	Use
<code>fopen()</code>	Opens a file
<code>fclose()</code>	Closes a file
<code>fprintf()</code>	Writes formatted data to a file
<code>fscanf()</code>	Reads formatted data from a file
<code>fgetc()</code>	Reads a single character
<code>fputc()</code>	Writes a single character
<code>fgets()</code>	Reads a string (line) from a file
<code>fputs()</code>	Writes a string to a file
<code>feof()</code>	Checks end of file
<code>rewind()</code>	Moves file pointer to beginning
<code>fseek()</code>	Moves file pointer to a specific position
<code>ftell()</code>	Returns current file pointer position

TWO MOST IMPORTANT 12 MARKS QUESTIONS (EXACT ANSWERS)

Likely to be asked again!

Q1. How to open and close an existing file in C?
Also discuss the various modes to open a file. (12 Marks)

Ans:

In C, a file is opened using the `fopen( )` function and closed using the `fclose( )` function. The `fopen( )` function returns a pointer of type `FILE` which is used to perform file operations. If the file cannot be opened, it returns `NULL`.

Syntax:

```
FILE *fp;
fp = fopen ("filename", "mode");
```

Example:

```
FILE *fp;
fp = fopen ("data.txt", "r");
if (fp == NULL)
{
    printf("Cannot open file\n");
}
else
{
    printf("File opened successfully\n");
}

fclose (fp); // Closing the file
```

File Opening Modes:

Mode	Purpose
<code>r</code>	Open for reading (file must exist)
<code>w</code>	Open for writing (creates new file / overwrites)
<code>a</code>	Open for appending (adds at end)
<code>r+</code>	Open for both reading and writing
<code>w+</code>	Open for both reading and writing (new file)
<code>a+</code>	Open for both reading and appending
<code>rb</code> , <code>wb</code> , <code>ab</code>	Binary mode (read / write / append)
<code>rb+</code> , <code>wb+</code> , <code>ab+</code>	Binary mode (read & write)

Q2. Write a program to copy the contents of one file to another file. (12 Marks)

Ans:

```
#include <stdio.h>

int main()
{
    FILE *src, *dest;
    char ch;

    src = fopen ("source.txt", "r");
    if (src == NULL)
    {
        printf ("Cannot open source file\n");
        return 1;
    }

    dest = fopen ("destination.txt", "w");
    if (dest == NULL)
    {
        printf ("Cannot open destination file\n");
        fclose (src);
        return 1;
    }

    while ((ch = fgetc (src)) != EOF)
    {
        fputc (ch, dest);
    }

    fclose (src);
    fclose (dest);

    printf ("File copied successfully\n");
    return 0;
}
```

Most Repeated 12-Mark Questions (Unit 7)

Q. No.	Question	Year Asked (AKU)
1	How to open and close a file in C? Discuss various file opening modes.	2015, 2016, 2018, 2020, 2022, 2024
2	Write a program to copy the content of one file to another file.	2016, 2017, 2019, 2021, 2023
3	Write a program to count number of characters/words/lines in a text file.	2017, 2020, 2022, 2024
4	Explain various file handling functions in C with example.	2018, 2021
5	Write a program to read and write student records using file (structure).	2019, 2023
6	Differentiate between text file and binary file.	2015, 2018, 2022

Quick Revision Points

1. A file is a collection of data stored on secondary storage.
2. FILE *fptr; is a file pointer (defined in stdio.h).
3. fopen() is used to open a file.
4. fclose() is used to close a file.
5. File opening modes:
 - r – open for reading (file must exist)
 - w – open for writing (creates new file)
 - a – open for appending
 - r+ – open for reading and writing
 - w+ – open for reading and writing (creates new file)
 - a+ – open for reading and appending
6. Common functions:
 - fopen(), fclose(), fprintf(), fscanf()
 - fgetc(), fputc(), fgets(), fputs()
 - feof(), rewind(), remove(), rename()
7. Text file – stores data in readable form.
8. Binary file – stores data in binary form (compact, faster).
9. Use structures with files to store multiple records.

Q.1

(12 Marks)

How to open and close an existing file in C?
Also discuss the various modes to open a file.

Ans : In C, file handling is performed using a file pointer of type FILE. The function fopen() is used to open a file and it returns a pointer to the file. The function fclose() is used to close a file.

1. Opening a file :

A file is opened using fopen() function.

Syntax : FILE *fptr;
fptr = fopen("filename", "mode");

If the file is successfully opened, fopen() returns a pointer to the file, otherwise it returns NULL.

2. Closing a file :

A file is closed using fclose() function.

Syntax : fclose(fptr);

It flushes the buffer and releases the file from memory.

3. Modes to open a file :

Mode	Meaning	Description
r	Read	Open for reading. File must exist.
w	Write	Open for writing. Creates new file. If file exists, its content is deleted.
a	Append	Open for appending. Writes at the end of file.
r+	Read and Write	Open for both reading and writing. File must exist.
w+	Read and Write	Creates new file for both reading and writing. If file exists, its content is deleted.
a+	Read and Append	Open for both reading and appending. Creates new file if it does not exist.

Q.2

(12 Marks)

Write a program to copy the
content of one file to another file.

Ans :

```
#include <stdio.h>

int main()
{
    FILE *fs,*fd;
    char ch;

    fs = fopen("source.txt", "r");
    if(fs == NULL)
    {
        printf("Cannot open source file\n");
        return 0;
    }

    fd = fopen("dest.txt", "w");
    if(fd == NULL)
    {
        printf("Cannot open destination file\n");
        fclose(fs);
        return 0;
    }

    while((ch = fgetc(fs)) != EOF)
    {
        fputc(ch, fd);
    }

    fclose(fs);
    fclose(fd);
    printf("File copied successfully.\n");
    return 0;
}
```

Exam Tips :

- ✓ Write points and syntax as per AKU pattern.
- ✓ Draw mode table for full marks.
- ✓ For programs, use proper variable names, indentation and messages.
- ✓ Practice previous year questions (2015–2024).
- ✓ In 12 marks, write explanation + syntax + example/code.

*"Practice Smart
Score High"*

**AKU BCA
2nd Semester**

All the Best!

MOST REPEATED 12 MARKS QUESTIONS FROM UNIT 7

S.No.	Question	Year Asked
1.	How to open and close a file in C? Also discuss the various modes to open a file.	2015, 2016, 2018, 2020, 2023
2.	Write a program to read and display the contents of a file (or copy one file into another file).	2016, 2019, 2021, 2024
3.	Write a program to merge two files.	2018, 2022
4.	Write a program to count the number of characters/ words/lines in a file.	2019, 2023

UNIT 7 – QUICK POINTS

- Header file : `stdio.h`
- File pointer : `FILE *fp;`
- Open file : `fopen()`
- Close file : `fclose()`
- Modes : `r, w, a, r+, w+, a+`
- Character I/O : `fgetc()`, `fputc()`
- String I/O : `fgets()`, `fputs()`
- Formatted I/O : `fprintf()`, `fscanf()`
- End of file : `feof()`
- Other functions : `rewind()`, `fseek()`, `ftell()`, `remove()`, `rename()`

Q1 (12 Marks)

How to open and close a file in C? Also discuss the various modes to open a file.

(Asked in 2015, 2016, 2018, 2020, 2023)

Answer:

In C, file handling is performed using a file pointer of type `FILE`. A file is opened using `fopen()` and closed using `fclose()`. The function `fopen()` returns a pointer to the file, which is used for all file operations. If the file cannot be opened, it returns `NULL`. The function `fclose()` is used to close the file and free the resources.

Syntax:

```
FILE *fp;
fp = fopen("filename", "mode");
```

```
fclose(fp);
```

Modes to open a file:

Mode	Description
<code>r</code>	Open an existing file for reading.
<code>w</code>	Open a file for writing (creates new file or truncates existing file).
<code>a</code>	Open a file for appending (writes at end).
<code>r+</code>	Open an existing file for both reading and writing.
<code>w+</code>	Open a file for both reading and writing (creates new file or truncates existing file).
<code>a+</code>	Open a file for both reading and appending (writes at end).

Example:

```
#include <stdio.h>
int main() {
    FILE *fp;
    fp = fopen("data.txt", "r");
    if (fp == NULL) {
        printf("Cannot open file\n");
        return 1;
    }
    /* file operations */
    fclose(fp);
    return 0;
}
```

Q2 (12 Marks)

Write a program to read and display the contents of a file.

(Asked in 2016, 2019, 2021, 2024)

Answer:**Program:**

```
#include <stdio.h>
int main() {
    FILE *fp;
    char ch;
    fp = fopen("data.txt", "r");
    if (fp == NULL) {
        printf("Cannot open file\n");
        return 1;
    }
    printf("\nContents of the file are:\n");
    while ((ch = fgetc(fp)) != EOF) {
        putchar(ch);
    }
    fclose(fp);
    return 0;
}
```

Output:

If data.txt contains:

```
Hello
This is a file in C.
```

Output:

```
Contents of the file are:
Hello
This is a file in C.
```

Explanation:

- The file is opened in read mode using `fopen()`.
- `fgetc()` reads one character at a time until end of file (EOF).
- Each character is displayed using `putchar()`.
- The file is closed using `fclose()`.

EXAM TIPS:

- ✓ Write neat and clean programs.
- ✓ Explain functions briefly.
- ✓ Draw tables (like file modes) for extra marks.

STUDY SMART
SCORE HIGH!